MASTER OPERATING PLAN

The Helm

Rules, Risk Framework & Execution Standard

Markets: Micro Futures (MES / MCL primary; MNQ, MGC, ES, CL supported)
Timeframes: Intraday (5m / 15m / 30m / 1h) | Higher-TF Daily bias
Execution: Local LLM-assisted proposals + opt-in per-signal auto-execution
Operator: Single locked account

Version 1.1.4 (production / main) | Effective Date: 2026-06-17

LODESTONE & PURSER

1. Purpose & Mission

This Operating Plan governs all trading activity conducted through The Helm. It documents the system's rules, risk parameters, and execution standards exactly as they are implemented in code -- not as aspirations. The plan exists to make the process auditable: every guardrail named here is enforced by software, and every default is conservative by design.

1.1 Mission

To convert chart context into disciplined, structured trade proposals using a local vision-LLM, and to execute only the proposals a human deliberately arms. The system never decides to trade on its own. Discipline is mechanical: a proposal that fails a sanity check is dismissed automatically, and execution is gated behind a master switch that defaults OFF.

1.2 The Helm Is a Filter, Not a Trigger

The Helm produces proposals continuously, but most are **flat** -- the correct call when evidence conflicts or price sits at equilibrium with no anchor. A flat proposal is a successful screening pass, not a failed analysis. The system is built to do nothing when the market offers no structured edge, and to surface only setups that clear a reward-to-risk floor and a price-sanity check.

> **Design principle:** The bot proposes; the human disposes. Auto-execution is opt-in, per-signal, single-account, and Sim-capable by default. No autonomous order placement exists anywhere in the system.

1.3 Current Phase

Production version **1.1.4** on the `main` branch. Confidence scoring was removed in this generation; proposals are single-pass with no retry loop. The Auto-Trader (`HelmAutoTrader.cs`) is code-complete and account-locked, validated in Sim/Playback. The primary objective at this stage is process fidelity and signal-quality measurement, not P&L maximization.

1.4 Plan Hierarchy

Where operator judgment conflicts with an enforced rule, the enforced rule governs -- the code will reject the action regardless. Configurable guardrails (daily-loss cutoff, balance floor, max concurrent) may be tuned in Settings, but tightening is always permitted and loosening is a deliberate, logged act.

2. Core Operating Parameters at a Glance

All values below are read from the live configuration (`~/.helm/settings.json` , `instruments.json` , and the Pydantic settings schema). Defaults are conservative; the operator may tighten any limit.

Parameter	Value	Notes
Primary instruments	MES, MCL	MNQ, MGC, ES, CL and ~20 others supported via `instruments.json`
Auto-Trader account	Single locked account	Enforced by both dashboard and strategy (defense-in-depth)
Master switch	OFF (default)	Must be deliberately enabled; no autonomous execution
Max concurrent trades	1 per instrument	Global ceiling via `max_concurrent` (default 1)
Max contracts / order	2 (default)	Arm rejected (409) if ATM template size exceeds this
Minimum reward:risk	2:1	Enforced in prompt; lower-RR ideas forced flat
Max price drift	5%	Proposal auto-dismissed if entry/stop/target >5% off market
Entry window	240 min (4 hr)	Unfilled resting limit auto-cancels; disarm 60s before expiry
Daily loss cutoff	$0 = off (default)	When set & hit: new entries blocked, signals disarmed (positions held)
Min account balance	$0 = off (default)	When breached: master switch forced OFF (passive, no flatten)
Trading-day roll	5:00 PM CT	CME Globex session boundary; all "today" aggregates use it
Stale-bar threshold	120 seconds	Older bars skipped; first bar after 30+ min gap skipped
Auto-Trader poll	3 seconds (default)	Frequency of `/api/exec/queue` checks
Signal retention	7 days (default)	Old feed bars/signals pruned after this

> **Position sizing:** The Helm has *no* automatic contract-sizing formula. Quantity is fixed by the selected ATM template's bracket definition. Risk per trade is therefore governed by (template stop distance x

> instrument point value x leg quantity), and capped at the order level by `max_contracts_per_order`. The operator sizes risk by choosing the template.

3. Risk Management Rules

Risk management is enforced in code, not by convention. A proposal or order that violates a MANDATORY rule below is rejected by the system regardless of how attractive the setup looks.

#	Rule	Description	Category
01	**2:1 reward:risk floor**	Every directional proposal must clear a 2:1 reward-to-risk ratio. If no ATM template satisfies it given the reasoned stop, the proposal is forced flat.	**MANDATORY**
02	**Price-drift sanity**	Entry, stop, and target must each sit within 5% of current market price. Any drift beyond that auto-dismisses the proposal with a logged reason (`MAX_PRICE_DRIFT = 0.05`).	**MANDATORY**
03	**ATM template required**	A directional proposal with a blank `atm_strategy` field is auto-dismissed. Stop/target prices are derived from the template's tick offsets, never invented by the LLM.	**MANDATORY**
04	**Max 1 trade / instrument**	An instrument with an open or pending trade blocks all new signals for that instrument across every timeframe. Scale-out runner legs must all be closed before it frees.	**MANDATORY**
05	**Global concurrency cap**	`max_concurrent` caps total simultaneous instruments. Example: cap 2 allows one MES and one MCL trade at once, no more.	**MANDATORY**
06	**Contract ceiling**	Arm is rejected (409 Conflict) if the template's total quantity exceeds `max_contracts_per_order` (default 2). Micro contracts only at current account scale.	**MANDATORY**
07	**Daily loss cutoff**	When configured and session realized P&L ≤ -cutoff: auto-trader forced OFF, new entries blocked, remaining signals disarmed. Open positions are *not* force-flattened (fail-safe is passive).	**MANDATORY**
08	**Minimum balance floor**	When live equity ≤ `min_account_balance` : master switch forced OFF; strategy refuses to run.	**MANDATORY**
09	**Account lock**	All auto-execution targets one operator-selected account. `HelmAutoTrader.cs` refuses to run if `Account.Name` does not match the locked account.	**MANDATORY**
10	**Prop-firm drawdown tracking**	Optional per-account trailing / daily drawdown and profit-target tracking, color-coded OK / WARN (75%) / BREACH. Informational halt signal; not an auto-flatten.	**OPTIONAL**

4. Trade Entry Rules

Entries are generated by a vision-LLM reading authoritative market context. The prompt enforces a four-step decision process; a proposal that cannot satisfy every step resolves to flat.

#	Rule	Description	Category
01	**Direction first, no long-bias**	Bullish requires HH/HL + price above MA + BOS; bearish requires LH/LL + price below MA + CHoCH. Conflicting evidence or equilibrium with no anchor resolves to **flat**.	**MANDATORY**
02	**Premium / discount location**	Buy only in the discount half (below range equilibrium); sell only in the premium half. Equilibrium with no nearby level forces flat.	**MANDATORY**
03	**Institutional resting entry**	Entry is a resting limit at an order block / sweep retrace / breakout retest -- away from last price. It may never fill if price does not return.	**MANDATORY**
04	**Structure-anchored stop & target**	Stop sits just beyond the invalidation zone (~0.4-0.6x ATR buffer past the swing); target is the next liquidity pool. Template tick offsets must reconcile with this.	**MANDATORY**
05	**Authoritative context is truth**	The LLM must use the numeric context block (bid/ask, multi-TF OHLC, EMA-90, ATR-14, Donchian, floor pivots, 3-lens BOS/CHoCH) as ground truth -- not re-read pixels off the chart.	**MANDATORY**
06	**HTF alignment**	Higher-timeframe structure (30m / 1h / daily) is supplied in every context block; counter-trend entries must be justified against it.	**MANDATORY**
07	**Stale / post-gap guard**	Auto-analysis skips bars older than 120s and skips the first bar after a 30+ minute gap (avoids chaotic session opens). Minimum 20 bars of history required.	**MANDATORY**
08	**Active-trade gate**	Headless analysis skips any instrument that already has an open/pending trade, preventing signal stacking.	**MANDATORY**
09	**Single-pass, no confidence floor**	As of v1.1.4 there is no confidence threshold or retry loop. Every proposal -- flat or directional -- lands unless it fails a sanity check.	**CURRENT**

4.1 Two Entry Paths

Manual (hotkey): Operator presses Ctrl+Shift+F on a focused NinjaTrader chart. HelmFeed captures a cropped screenshot plus rich context and POSTs to `/api/capture-from-nt`; a proposal card appears on the Signal Analysis page within ~1s (warm) to ~30s (cold).

Automated (headless, bar-driven): Up to 4 (Instrument, Period) slots configured on Home. Each live bar close publishes to `/api/feed/bar`; if all gates pass (provider configured, no open trade, ≥20 bars, fresh bar,

no recent gap) the analyzer fires automatically.

5. ATM Templates & Bracket Structure

The Helm does not hardcode stop/target distances. It dynamically loads ATM templates from NinjaTrader's template folder on every analysis call and presents the matching ones as a scoped menu to the LLM. The template alone defines quantity, stop, target, breakeven, and trail behavior.

5.1 Naming Convention

`{INSTRUMENT}_{STYLE}_{SIZE}c_{STOP}-{TARGET}` -- e.g. `MES_INTRA_2c_10-25` = MES, intraday, 2-contract scale-out, 10-tick stop, 25-tick target. Templates outside the convention or folder are invisible to the system.

Example template	Qty	Stop	Target	R:R	Use
`MES_SCALP_1c_6-15`	1	6 t	15 t	2.5:1	Index scalp
`MES_INTRA_2c_10-25`	2	10 t	25 t	2.5:1	Index intraday scale-out
`MCL_SCALP_1c_8-20`	1	8 t	20 t	2.5:1	Crude scalp
`MCL_INTRA_2c_15-30`	2	15 t	30 t	2:1	Crude intraday scale-out

Templates are operator-defined and live in the NinjaTrader XML folder; the table above is representative, not exhaustive. The system adapts to whatever valid templates exist.

5.2 Scale-Out Legs

- **TP1 (first bracket):** tightest target, auto-breakeven at a profit trigger, plus trail steps.
- **Runner (second+ bracket):** wider target, trails independently; stays open after TP1.
- Per-leg P&L is computed independently; the aggregate outcome is final only when all legs close.
- Scale-out fills are stored at `signal.legs[]` (top level), not nested under outcome.

5.3 Trail Mechanics

- **Auto-breakeven:** at `AutoBreakEvenProfitTrigger` ticks of profit, stop moves to entry + `AutoBreakEvenPlus`.
- **Trail steps:** at each `ProfitTrigger`, stop moves to current price minus `StopLoss` ticks.
- Trail is executed by NinjaTrader's ATM engine per-leg; The Helm reads the realized exit from NT8's SQLite.

> **LLM's role is limited:** it picks an exact template name from the scoped menu and supplies an entry price. Stop and target are derived from the template offsets + entry + direction. The LLM cannot invent a custom bracket.

6. Trade Exit & Outcome Resolution

Exits are managed by NinjaTrader's ATM engine; outcomes are resolved automatically by an independent tick-walk watcher and recorded against the real broker fills.

#	Rule	Description	Category
01	**Stop is structural**	The stop is placed at the invalidation level at entry. ATM trail/breakeven only move it toward profit -- never wider.	**MANDATORY**
02	**Scale at TP1**	First bracket exits at its target; remaining runner continues under trail or a wider target. Defined entirely by the chosen template.	BY TEMPLATE
03	**Trail the runner**	Runner trails per the template's `trail_steps[]`; breakeven and step logic are pre-defined before entry, never improvised mid-trade.	BY TEMPLATE
04	**Automatic outcome walk**	`outcome_watcher.py` polls `feed.db` every 30s: did entry fill? then which hit first, target or stop? It stamps `outcome.result` = target / stop / breakeven / partial / no_fill.	AUTOMATIC
05	**Entry/outcome invariant**	Code enforces `outcome=no_fill` ⇔ `entry_triggered=False`. Any other outcome implies the entry filled. Manual edits coerce the pair on every write.	**MANDATORY**
06	**Entry window expiry**	Unfilled resting limit cancels after 240 min; the strategy disarms it 60s before expiry. Unfilled = `no_fill`.	**MANDATORY**
07	**Ground-truth P&L**	Realized P&L is derived from actual NT8 fills in `trades.db` (recorded independently by `recorder.py`), not from a simulated estimate.	**MANDATORY**
08	**Manual override**	The Signal Detail page exposes an outcome editor (result dropdown + optional closing price) for cases the watcher cannot resolve.	OPERATOR

7. Auto-Execution Standard

Auto-execution is the most safety-critical subsystem and is therefore the most heavily gated. It is opt-in at three independent levels and defaults fully OFF.

7.1 Three-Level Opt-In

- **Global enable** -- master switch in Settings, default **OFF**.
- **Account lock** -- a single NT account ID; trades place only there, enforced by both dashboard and `HelmAutoTrader.cs`.
- **Per-signal arm** -- the operator manually arms each proposal; only armed signals enter the execution queue.

7.2 Execution Flow

- Operator arms a signal -> `armed=True` on the locked account.
- `HelmAutoTrader.cs` polls `/api/exec/queue?account=<locked>` every 3s; the queue returns only armed signals for instruments with no open trade, and only while the master switch is on and no fail-safe has tripped.
- Strategy places `AtmStrategyCreate(Limit@entry, template_name)`; quantity is fixed by the template.
- State machine advances `armed -> working -> filled`; `exec_tag` (`helm_<ts>`) links the proposal to the real fills.

7.3 Guardrails (all enforced)

Guardrail	Behavior
Dry-run mode (default ON)	Logs "WOULD place..." without touching the order book; flip OFF to go live
Account lock	Strategy refuses to run on any account but the locked one
Max concurrent	One open trade per instrument; checked before placing
Contract ceiling	Arm rejected if template qty > `max_contracts_per_order`
Entry window	240 min; cancel + disarm 60s before expiry
Daily loss cutoff	Blocks new entries + disarms remaining signals when hit
Min balance floor	Forces master switch OFF when breached (passive)
Thread safety	ATM calls marshalled to the strategy thread via `TriggerCustomEvent`

> **Forever rule:** The Helm performs no autonomous execution. The human arms every signal explicitly; the strategy only places the named ATM template the human selected. The master switch always defaults OFF.

8. Signal & Trade Journaling

Every proposal is persisted automatically -- journaling is not a manual chore but a property of the pipeline. The append-only record is the basis for measuring signal quality.

8.1 Captured Per Signal (`signals.jsonl`)

- Timestamp (unix ns), instrument, direction, and the full proposal (entry / stop / target / ATM template).
- Cropped chart screenshot at capture (`screenshot_path`).
- Full authoritative market context block (multi-TF OHLC, EMA-90, ATR-14, Donchian, pivots, 3-lens BOS/CHoCH).
- Provider and model used (e.g. `qwen2.5vl:7b` via Ollama).
- Operator journal: verdict + free-text note.
- Outcome record (result, entry_triggered, entry_hit_ts) and scale-out `legs[]` .
- Execution record (state, placed_at, filled_at, exec_tag) once armed.
- Auto-dismissal flag + reason when a sanity check rejects a proposal.

8.2 Real Fills (`trades.db`)

Recorded independently from NinjaTrader's own SQLite by `recorder.py` : account, instrument, quantity, entry/exit fills, commissions, and real net P&L. This is the ground truth the Trade Performance page and the outcome watcher rely on.

8.3 Storage & Recovery

- `signals.jsonl` is append-only; updates append a new line with the same ts and the reader merges latest-wins.
- Soft-delete via a `deleted:true` line (recoverable by hand).
- Feed history (`feed.db`) auto-prunes after the retention window (default 7 days).
- Secrets never enter these files -- API keys and broker account IDs live only in git-ignored `~/.helm/credentials.json` .

9. Review & Metrics

The dashboard surfaces the review metrics continuously rather than at a fixed meeting cadence. Review is the act of reading these against the trading-day boundary.

9.1 Pages

Page	Purpose
Home	Current CME-session snapshot, cumulative P&L by bucket, action queue, drawdowns, economic calendar, auto-analysis status
Signal Analysis	Filterable proposal table + signal detail (screenshot, reasoning, journal, outcome editor, arm/disarm)
Trade Performance	Real round-trip trades, aggregate stats, equity curve, breakdowns by symbol / account / ATM template
Health	API status, feed-latency percentiles, last 50 log lines
Settings	AI backend, account bucketing, strategy thresholds, auto-trader controls, timezone, news filters, data-integrity auditor

9.2 Metrics Tracked

- Win rate and realized R per round-trip (from real fills).
- Realized P&L aggregated to the CME trading-day boundary.
- Outcome distribution (target / stop / breakeven / partial / no_fill).
- Drawdown status vs. trailing / daily / profit-target limits (prop-firm tracking).
- Proposal volume vs. auto-dismissal rate (signal-quality proxy).
- Feed latency and service health.

9.3 Session Timing

- **Trading-day roll:** 5:00 PM CT (`ROLL_HOUR = 17`), converted to the operator timezone. All "today" filters use this, not calendar midnight.
- **Globex:** 5:00 PM CT Sunday -> 4:00 PM CT Friday, with a daily 4:15-5:00 PM CT maintenance halt that auto-analysis skips.
- **News context:** high-impact USD events scoped to the current trading-day window, refreshed every 15 min (5-180 configurable).

10. Trading Universe

The Helm trades micro futures via NinjaTrader. The instrument map in `instruments.json` is authoritative for tick size, point value, and commission.

10.1 Primary & Secondary Instruments

Contract	Name	Tick	Point Value	Comm/RT	Tier
MES	Micro E-mini S&P 500	0.25	$5.00	$1.10	Primary
MCL	Micro Crude Oil	0.01	$100.00	$1.15	Primary
MNQ	Micro E-mini Nasdaq-100	0.25	$2.00	$1.10	Primary
MGC	Micro Gold	0.10	$10.00	$1.15	Secondary
ES	E-mini S&P 500	0.25	$50.00	$3.18	Supported
CL	Crude Oil	0.01	$1,000.00	$6.00	Supported
NQ	E-mini Nasdaq-100	0.25	$20.00	$3.18	Supported
GC	Gold	0.10	$100.00	$5.00	Supported

Additional supported symbols (YM, MYM, RTY, M2K, SI, HG, NG, ZB, ZN, 6E, 6B, 6J, 6A, BTC, MBT, ETH, etc.) carry full tick/point maps in `instruments.json`. Only micro contracts are intended at current account scale.

10.2 Timeframes

Auto-analysis slots target 5m and 15m execution timeframes; the context block always carries 5m / 15m / 30m / 1h / daily for higher-timeframe bias. Structure is read at three retrace-sensitivity lenses (50% / 100% / 200%).

11. System Architecture

11.1 Topology

- **Trading host (GEEKOM):** Windows 11; runs NinjaTrader 8, the FastAPI dashboard (uvicorn on 127.0.0.1:8000, loopback only), HelmFeed + HelmAutoTrader NinjaScript, and the recorder.
- **Inference backend (workstation, optional):** Ubuntu 24.04, RTX 4060 Ti 16GB, Ollama on the LAN (UFW-restricted to the trading host). Default model `qwen2.5vl:7b` (Q4), fallback `minicpm-v`.
- **Service:** uvicorn runs under an NSSM watchdog (`HelmDashboardWatchdog`) that auto-restarts on crash.

11.2 NinjaScript

- **HelmFeed.cs** -- single indicator on every analyzed chart. Publishes bars + ticks + screenshots, builds the context block, and serves the Ctrl+Shift+F hotkey path.
- **HelmAutoTrader.cs** -- one strategy instance per instrument. Polls the exec queue, places account-locked ATM entries, reports live equity for the balance fail-safe.

11.3 Data Stores

Store	Contents
`signals.jsonl`	Append-only proposal log (proposal, context, journal, outcome, legs, exec)
`trades.db`	Real NT8 fills mirror -- ground-truth P&L
`feed.db`	Live bars + ticks for context + staleness checks (auto-pruned)
`~/.helm/settings.json`	Non-sensitive operator config (UI-writable)
`~/.helm/credentials.json`	API keys + broker account IDs (git-ignored, never overwritten)

12. Versioning & Change Control

12.1 Semantic Versioning

- **MAJOR** -- breaking change to signal schema, settings shape, NS⇔bot API, or ATM template format.
- **MINOR** -- backward-compatible feature (new router, new setting with default).
- **PATCH** -- backward-compatible bug fix.
- **Pre-release** -- `-beta.N`, never a production release.

12.2 Channels

Channel	Branch	Tag	Auto-pull	Role
Production	`main`	`vX.Y.Z`	Yes (version-check)	Running bot
Beta	`beta`	`vX.Y.Z-beta.N`	No	Manual validation

Current production: **1.1.4**. Never push unvalidated work to `main`; never run the in-app updater while on the beta branch.

12.3 In-App Updater

Support > Update (or the update banner) runs a detached `update.ps1`: `git fetch && git reset --hard origin/main`, conditional pip/npm reinstall, always rebuild the React bundle, then bounce uvicorn (the watchdog respawns it). NinjaScript changes still require a manual F5 compile in the NS editor.

12.4 Change Control

This document is the operating snapshot of an evolving system. Material changes to risk guardrails, the entry decision process, or the auto-execution standard should be reflected here and version-stamped. Tightening a guardrail is always safe; loosening one is a deliberate, logged decision.

Date	Area	Change	Version